

Guion de Presentación

Patrón Decorator — Lenguajes de Programación II

Imprimir y usar como apoyo durante la exposición.

SLIDE 1

Título

Hoy les voy a explicar el patrón de diseño **Decorator**, uno de los patrones estructurales del catálogo Gang of Four.

Su objetivo es añadir comportamiento a un objeto en tiempo de ejecución, sin modificar su código original. Al final van a entender cómo funciona, por qué existe y cómo se usa.

SLIDE 2

El problema

Imaginemos que tenemos un sistema que necesita notificar por Email, SMS y Slack.

Si lo resolvemos con herencia, necesitamos una clase por cada combinación posible: Email, SMS, Slack, EmailSms, EmailSlack, SmsSlack, EmailSmsSlack. Son **7 clases para solo 3 canales**. Si agregamos WhatsApp mañana, saltamos a 15.

Eso se llama **explosión combinatoria**. La herencia es estática — la combinación se decide al compilar y no puede cambiar.

Con Decorator resolvemos todo con **4 clases** y combinaciones ilimitadas en tiempo de ejecución.

NOTE: Señalar la columna izquierda (herencia) y la derecha (Decorator) en la slide.

SLIDE 3

Las 4 clases del diseño

El diseño tiene exactamente 4 piezas.

Notifier: la interfaz. Define el contrato: un solo método `send()`. Todo el sistema habla este idioma.

BaseNotifier: el núcleo vacío. Implementa `Notifier` pero su `send()` no hace nada. Existe para que ningún canal sea obligatorio — es el punto de arranque neutral de cualquier cadena.

BaseDecorator: clase abstracta. Guarda la referencia al objeto envuelto en el campo `wrapped` y delega `send()` hacia él. Es la única clase que conoce el mecanismo de wrapping.

EmailDecorator, SmsDecorator, SlackDecorator: los decoradores concretos. Heredan `BaseDecorator`, llaman a `super.send()` y añaden su canal.

NOTE: Señalar el flujo de flechas entre las 4 piezas en la slide.

SLIDE 4

El código completo

El código tiene dos columnas.

A la izquierda están las tres bases: `Notifier` como interfaz, `BaseNotifier` con el cuerpo vacío, y `BaseDecorator` con el campo `wrapped` y la delegación.

A la derecha están los tres decoradores concretos. Noten que todos siguen exactamente el mismo patrón: constructor con `super(w)`, y `send()` que llama a `super.send()` antes de imprimir su canal.

La simetría entre los tres decoradores es intencional y es una señal de buen diseño: agregar un cuarto canal como WhatsApp sería copiar esa estructura y cambiar una sola línea.

SLIDE 5

Composición libre en el main

Aquí es donde el patrón muestra su valor. Podemos armar cualquier combinación encadenando constructores.

Solo Email: `new EmailDecorator(new BaseNotifier())`. El Email ya no es obligatorio — es un canal opcional como cualquier otro.

Solo SMS: mismo esquema pero con `SmsDecorator`. Sin email, sin Slack.

SMS + Slack sin Email: apilamos dos decoradores sobre `BaseNotifier`. El email no aparece en ningún lado.

Los tres juntos: apilamos los tres decoradores. Una sola llamada a `send()` dispara los tres canales en orden.

NOTE: Destacar que el tipo de todas las variables es `Notifier` — el caller nunca sabe cuántas capas hay.

SLIDE 6

Cierre

Para cerrar, tres frases para recordar las piezas:

"BaseNotifier es el punto de partida vacío. BaseDecorator es el portero. Cada decorador concreto es un canal que se activa apilándolo."

Los cuatro puntos clave: Notifier es el contrato que une a todos; BaseNotifier elimina la dependencia obligatoria del email; BaseDecorator centraliza el mecanismo de wrapping; los decoradores concretos solo añaden su canal con `super.send()`.

Cualquier pregunta, con gusto la respondo.

Posibles preguntas del docente

■ ¿Por qué BaseNotifier tiene el `send()` vacío y no lanza una excepción?

Porque su propósito es ser un punto de arranque neutral, no un contrato que deba cumplirse. Si lanzara una excepción, ningún decorador podría funcionar solo — siempre habría que terminar la cadena con un componente real. El cuerpo vacío permite que cada canal sea completamente opcional.

■ ¿Qué diferencia hay entre BaseNotifier y BaseDecorator?

`BaseNotifier` es un componente concreto: implementa `Notifier` directamente y no envuelve a nadie. Es la hoja del árbol. `BaseDecorator` es abstracto: también implementa `Notifier` pero su propósito es envolver a otro `Notifier` y delegar. Es el nodo interno del árbol. Ningún decorador funciona sin un componente al final de la cadena.

■ **¿Qué pasa si cambio el orden de los decoradores?**

El orden importa. El decorador más interno ejecuta primero. Si usas `new SmsDecorator(new EmailDecorator(new BaseNotifier()))`, el Email se imprime antes que el SMS. La salida siempre sigue el orden de apilado de adentro hacia afuera.

■ **¿Cuál es la diferencia entre Decorator y herencia en este caso?**

Con herencia, la combinación de canales se decide en tiempo de compilación y genera una clase por combinación — explosión combinatoria. Con Decorator, la combinación se decide en tiempo de ejecución encadenando constructores. N canales requieren N decoradores, no 2^N clases.